

Tugas 3

Sistem Paralel dan Terdistribusi A

Implementasi Distributed Synchronization System



Disusun Oleh :

Olivia Dafina 11231077

02 Mei 2026



GITHUB

<https://github.com/oliviadafina/distributed-sync-system>

YOUTUBE

<https://youtu.be/rrKOIAeSnCo>

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi komputasi modern telah mendorong arsitektur sistem dari model *single-server* tradisional menuju sistem terdistribusi yang melibatkan banyak node yang saling berkomunikasi. Dalam lingkungan terdistribusi, tantangan utama yang muncul bukan lagi sekadar kecepatan pemrosesan, melainkan bagaimana menjamin konsistensi data, koordinasi antar proses, dan ketahanan sistem ketika salah satu komponen mengalami kegagalan.

Masalah sinkronisasi menjadi sangat krusial dalam konteks ini. Ketika dua atau lebih node mencoba mengakses dan memodifikasi sumber daya yang sama secara bersamaan, dapat terjadi *race condition* yang mengakibatkan data tidak konsisten. Selain itu, ketika salah satu node mati, sistem harus mampu melanjutkan operasi tanpa kehilangan data atau mengalami *downtime* yang berkepanjangan.

Berbagai algoritma dan protokol telah dikembangkan untuk mengatasi masalah-masalah tersebut. Raft Consensus Algorithm hadir sebagai solusi untuk pemilihan koordinator (*leader*) dan replikasi log secara konsisten. Consistent Hashing digunakan untuk mendistribusikan beban data secara merata tanpa memerlukan rekonfigurasi besar saat node ditambah atau dihapus. Sementara itu, MESI Protocol yang berasal dari dunia arsitektur prosesor diadaptasi untuk menjaga koherensi *cache* di seluruh node terdistribusi.

1.2 Tujuan

1. Mengimplementasikan Distributed Lock Manager menggunakan algoritma Raft Consensus dari nol (*from scratch*) dengan Python, yang mampu menangani *exclusive lock* dan *shared lock* antar banyak node secara konsisten.
2. Membangun Distributed Queue System berbasis Consistent Hashing yang mendukung *at-least-once delivery* dan berjalan lintas node tanpa kehilangan data.
3. Mengimplementasikan Cache Coherence Protocol MESI pada sistem multi-node dengan kebijakan eviksi LRU dan monitoring performa secara *real-time*.

- 
4. Mengemas seluruh sistem dalam container Docker menggunakan Docker Compose untuk kemudahan deployment dan skalabilitas.
 5. Sebagai fitur bonus, mengimplementasikan Role-Based Access Control (RBAC) dengan audit logging untuk keamanan akses antar node.

1.3 Ruang Lingkup

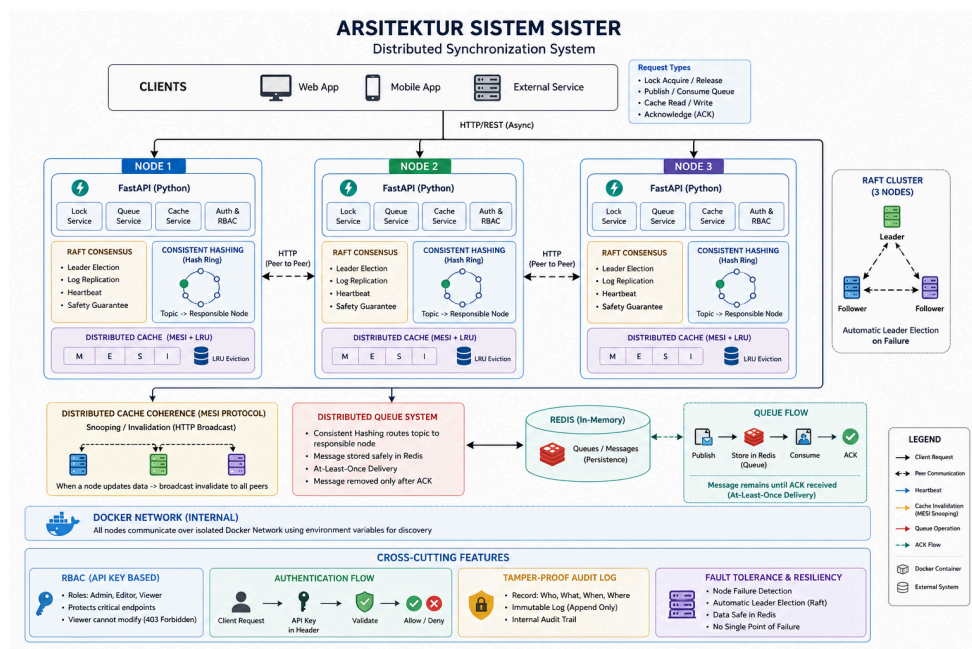
Sistem yang dibangun dalam tugas ini terdiri dari tiga node server independen yang berjalan dalam container Docker terpisah, satu instance Redis sebagai **shared state**, dan satu instance Locust untuk pengujian performa. Seluruh komunikasi antar node dilakukan melalui protokol HTTP menggunakan FastAPI sebagai **web framework**. Implementasi algoritma Raft, Consistent Hashing, dan MESI Protocol dilakukan secara manual tanpa menggunakan **library** pihak ketiga untuk komponen inti.

BAB 2

ARSITEKTUR SISTEM

2.1 Gambaran Umum

Sistem yang dibangun merupakan kluster terdistribusi yang terdiri dari tiga node server independen dan satu instance Redis sebagai *shared memory*. Setiap node berjalan di dalam container Docker yang terpisah dan berkomunikasi satu sama lain secara eksklusif melalui protokol HTTP. Dengan desain ini, tidak ada *shared memory* antar proses seluruh koordinasi dilakukan melalui pesan yang dikirim lewat jaringan. Diagram arsitektur sistem secara keseluruhan ditunjukkan pada gambar berikut:



Gambar 2.1 Diagram Arsitektur Sistem

2.2 Arsitektur Node

Setiap node menjalankan satu FastAPI web server yang berperan ganda:

1. Public Interface, melayani request dari client (acquire lock, publish message, read/write cache)
2. Internal Interface, melayani komunikasi antar node untuk protokol Raft (`/raft/request_vote``, `/raft/append_entries``), bus snooping MESI (`/cache/snoop``), dan queue routing

Dengan arsitektur ini, setiap node adalah *peer* yang setara atau tidak ada node yang secara permanen berperan sebagai "master". Peran Leader pada Raft bersifat dinamis dan dapat berpindah secara otomatis bila node yang menjadi Leader mengalami kegagalan.

2.3 Komponen Utama

Sistem terdiri dari tiga komponen inti yang berjalan secara bersamaan pada setiap node:

Komponen	Algoritma	Peran
Distributed Lock Manager	Raft Consensus	Koordinasi akses eksklusif ke shared resource
Distributed Queue System	Consistent Hashing	Distribusi dan routing pesan antar node
Distributed Cache Coherence	MESI Protocol + LRU	Sinkronisasi data cache antar node

Tabel 2.1 Komponen Utama Sistem

2.4 Peran Redis

Redis berfungsi sebagai "Main Memory" bersama untuk seluruh node. Redis menyimpan:

1. State antrian, daftar pesan yang menunggu dikonsumsi (Redis List)
2. Data cache yang di-flush, data dari node yang mengalami eviction atau invalidasi (MESI state M)
3. Pesan dalam pemrosesan, pesan yang sudah di-dequeue namun belum di-ACK, untuk keperluan recovery

Redis dijalankan dengan konfigurasi AOF (Append-Only File) sehingga data tetap tersimpan meskipun Redis di-restart.

2.5 Stack Teknologi

Layer	Teknologi	Versi
Web Framework	FastAPI	0.110
ASGI Server	Uvicorn	0.29
HTTP Client (inter-node)	HTTPX	0.27
Distributed State	Redis	7 (Alpine)
Data Validation	Pydantic	v2
Containerization	Docker + Docker Compose	-
Load Testing	Locust	2.24
Unit Testing	Pytest + pytest-asyncio	-

Tabel 2.2 Stack Teknologi

BAB 3

ALGORITMA

3.1 Raft Consensus Algoritma

Raft adalah algoritma konsensus yang dirancang untuk mudah dipahami dan diimplementasi. Tujuannya adalah memastikan semua node dalam kluster sepakat pada urutan perintah yang sama, meskipun ada node yang gagal di tengah jalan. Dalam sistem ini, Raft digunakan sebagai fondasi Distributed Lock Manager.

Setiap node dalam kluster Raft selalu berada di salah satu dari tiga state:

1. Follower, state awal semua node. Menerima dan mengikuti perintah Leader.
2. Candidate, transisi sementara saat node mencalonkan diri jadi Leader.
3. Leader, satu-satunya node yang boleh menerima perintah dari client. Mengirim heartbeat ke semua Follower.

Proses pemilihan Leader terjadi ketika Follower tidak menerima heartbeat dari Leader dalam batas waktu tertentu (election timeout). Prosesnya:

1. Follower mengubah state menjadi Candidate, menaikkan `current_term`, dan memilih dirinya sendiri.
2. Candidate mengirim pesan RequestVote ke semua peer secara paralel.
3. Node lain memberikan suara jika: belum memilih di term ini, dan log Candidate minimal sama lengkapnya dengan miliknya.
4. Jika Candidate mendapat suara mayoritas (≥ 2 dari 3 node), ia menjadi Leader.
5. Leader mulai mengirim heartbeat (AppendEntries kosong) setiap 500ms untuk mencegah election baru.

Untuk menghindari split vote, setiap node menggunakan randomized election timeout sehingga tidak terjadi pemilihan secara bersamaan.

Pada sistem ini, seluruh perintah seperti acquire dan release lock hanya diproses oleh Leader. Leader akan mendistribusikan perintah tersebut ke node lain melalui mekanisme AppendEntries.

Suatu log akan dianggap valid dan dapat dieksekusi apabila telah dikonfirmasi oleh mayoritas node. Hal ini memastikan bahwa seluruh node memiliki urutan eksekusi yang konsisten, sehingga tidak terjadi konflik data.

3.2 Consistent Hashing

Consistent Hashing merupakan metode distribusi data ke beberapa node dengan tujuan meminimalkan perpindahan data ketika terjadi perubahan jumlah node. Berbeda dengan metode hashing konvensional, teknik ini hanya memindahkan sebagian kecil data ketika node ditambah atau dihapus.

Dalam pendekatan ini, node ditempatkan pada sebuah struktur berbentuk cincin (hash ring) berdasarkan nilai hash. Untuk meningkatkan pemerataan distribusi data, setiap node direpresentasikan oleh beberapa virtual node. Data atau topik yang masuk akan dipetakan ke dalam cincin berdasarkan nilai hash, kemudian diarahkan ke node pertama yang memiliki nilai hash lebih besar atau sama.

Sistem mendukung mekanisme transparent routing, di mana client tidak perlu mengetahui lokasi node tujuan. Node yang menerima permintaan akan menentukan node yang bertanggung jawab, dan meneruskan permintaan tersebut jika diperlukan. Keuntungan dari pendekatan ini adalah fleksibilitas akses, karena client dapat berkomunikasi dengan node mana pun tanpa mempengaruhi hasil akhir.

3.3 MESI Cache Coherence Protocol

Setiap node dalam sistem memiliki cache lokal untuk meningkatkan performa akses data. Namun, hal ini dapat menyebabkan inkonsistensi data jika tidak dikelola dengan baik. MESI Protocol digunakan untuk menjaga konsistensi antar cache pada berbagai node.

Setiap data dalam cache memiliki empat kemungkinan state:

1. Modified (M), data telah diubah secara lokal dan belum disinkronkan ke penyimpanan utama.
2. Exclusive (E) data hanya terdapat pada satu node dan masih sesuai dengan sumber utama.
3. Shared (S), data dapat dimiliki oleh beberapa node dan masih konsisten.

- 
4. Invalid (I), data tidak valid dan tidak boleh digunakan.

Pada saat terjadi permintaan baca (read), sistem akan memeriksa apakah data tersedia di cache. Jika tidak tersedia, node akan melakukan broadcast ke node lain untuk memastikan status data, kemudian mengambil data dari penyimpanan utama dan menetapkan state yang sesuai. Pada saat terjadi penulisan (*write*), node akan mengirimkan sinyal ke node lain untuk menginvalidasi data yang sama. Setelah itu, data diperbarui pada node lokal dan ditandai sebagai *Modified*.

Cache memiliki kapasitas terbatas sehingga diperlukan mekanisme penggantian data. Sistem menggunakan metode Least Recently Used (LRU) untuk menentukan data yang akan dihapus. Jika data yang dihapus memiliki status Modified, maka data tersebut akan terlebih dahulu disimpan ke penyimpanan utama (write-back) sebelum dihapus dari cache, sehingga tidak terjadi kehilangan data.

3.4 Role-Based Access Control (RBAC)

Sebagai fitur tambahan, sistem ini mengimplementasikan mekanisme keamanan berbasis *Role-Based Access Control* (RBAC) menggunakan API Key.

Pembagian Role:

1. Admin, memiliki akses penuh terhadap seluruh operasi sistem, termasuk penulisan data dan manajemen lock.
2. Viewer, hanya memiliki akses untuk membaca data dan melakukan monitoring.

Setiap aktivitas pengguna dicatat dalam sistem log untuk keperluan audit dan keamanan.

BAB 4

DOKUMENTASI API

4.1 Gambaran Umum

Sistem yang dikembangkan menggunakan arsitektur *distributed system* di mana setiap node menyediakan layanan berbasis REST API. API ini dapat diakses melalui antarmuka dokumentasi interaktif Swagger UI pada alamat <http://localhost:800X/docs>

Setiap node memiliki base URL yang berbeda, sebagaimana ditunjukkan pada Tabel 4.1 Base URL berikut:

Node	Base URL
Node 1	http://localhost:8001
Node 2	http://localhost:8002
Node 3	http://localhost:8003

Tabel 4.1 Base URLs

Sebagian besar endpoint memerlukan autentikasi menggunakan API Key, kecuali endpoint tertentu seperti `/health` dan endpoint internal sistem.

4.2 Mekanisme Autentikasi

Sistem menggunakan pendekatan *Role-Based Access Control* (RBAC) berbasis API Key. Setiap request harus menyertakan header sebagai berikut:

`X-API-Key: admin-secret-key-123` atau `X-API-Key: viewer-secret-key-456`

Perbedaan hak aksesnya antara lain:

1. Admin: dapat melakukan seluruh operasi (read & write)
2. Viewer: hanya dapat melakukan operasi baca (*read-only*)

Jika autentikasi gagal, sistem akan memberikan respon seperti pada tabel dibawah ini

HTTP Code	Keterangan
401	Header tidak disertakan
403	API Key tidak valid / tidak memiliki izin

Tabel 4.2 Respon Autentikasi

4.3 Endpoint Health Check

Endpoint `GET /health` digunakan untuk mengetahui kondisi node serta status konsensus Raft. Endpoint ini tidak memerlukan autentikasi. Contoh Response:

```
{
  "status": "ok",
  "node_id": "node_3",
  "raft_state": "leader",
  "current_term": 1,
  "commit_index": 5
}
```

Endpoint ini penting untuk mengidentifikasi node yang berperan sebagai *leader* sebelum melakukan operasi tertentu seperti *lock*.

4.4 Distributed Lock API

Seluruh operasi lock hanya dapat dilakukan melalui node yang berstatus *leader*.

1. Acquire Lock

Dengan menggunakan endpoint `POST /lock/acquire`, contoh Request:

```
{  
  
  "resource": "database_master",  
  
  "client_id": "client_A",  
  
  "type": "exclusive",  
  
  "timeout": 10,  
  
  "ttl": 300  
  
}
```

Penjelasan Parameter:

- `resource` : nama resource yang dikunci
- `client_id` : identitas client
- `type` : jenis lock (exclusive/shared)
- `timeout` : waktu tunggu
- `ttl` : masa berlaku lock

Contoh Response Sukses:

```
{  
  
  "status": "success",  
  
  "message": "Lock acquired"  
  
}
```

2. Release Lock

Dengan menggunakan endpoint `POST /lock/release`, contoh Request:

```
{  
  
  "resource": "database_master",  
  
  "client_id": "client_A"  
  
}
```

Contoh Response:

```
{  
  
  "status": "success",  
  
  "message": "Lock released"  
  
}
```

4.5 Distributed Queue API

Sistem antrian menggunakan mekanisme *consistent hashing* sehingga client dapat mengirim request ke node mana pun.

1. Publish Message

Dengan menggunakan endpoint `POST /queue/publish`, contoh Request:

```
{  
  
  "topic": "pesanan",  
  
  "payload": {  
  
    "item": "Laptop",  
  
    "jumlah": 2  
  
  }  
  
}
```

Contoh Response:

```
{  
  
  "status": "success",  
  
  "message": {  
  
    "message_id": "uuid-xxx",  
  
    "topic": "pesanan"  
  
  }  
  
}
```

```
}
```

2. Poll Message

Dengan menggunakan endpoint `GET /queue/poll/{topic}`, contoh Response:

```
{
  "status": "success",
  "message": {
    "message_id": "uuid-xxx"
  }
}
```

3. Acknowledge Message

Dengan menggunakan endpoint `POST /queue/ack`, contoh Request:

```
{
  "topic": "pesanan",
  "message_id": "uuid-xxx"
}
```

4.6 Distributed Cache API

1. Write Cache

Dengan menggunakan endpoint `POST /cache/{key}`, contoh Request:

```
{
  "value": "50000000"
}
```

2. Read Cache

Dengan menggunakan endpoint `GET /cache/{key}`, contoh Response:

```
{
```

```
"status": "success",  
  
"key": "harga",  
  
"value": "50000000"  
  
}
```

3. Cache Metrics

Dengan menggunakan endpoint `GET /cache/metrics`, contoh Response:

```
{  
  
  "cache_size": 5,  
  
  "hits": 142,  
  
  "misses": 23  
  
}
```

4.7 Endpoint Internal

Endpoint berikut digunakan untuk komunikasi antar node dan tidak digunakan oleh client:

- `/raft/request_vote`
- `/raft/append_entries`
- `/cache/snoop`

Endpoint ini mendukung mekanisme konsensus dan sinkronisasi antar node dalam sistem.

BAB 5

PANDUAN DEPLOYMENT

5.1 Prasyarat

Sebelum menjalankan sistem, pengguna perlu memastikan bahwa perangkat telah memiliki beberapa perangkat lunak pendukung dengan versi minimum tertentu. Prasyarat ini diperlukan agar proses deployment dapat berjalan dengan lancar tanpa kendala kompatibilitas.

Software	Versi Minimum	Fungsi
Docker Desktop	20.0+	Menjalankan kontainer node dan Redis
Docker Compose	2.0+	Mengelola dan mengorkestrasi multi-container
Python	3.8+	Menjalankan unit testing secara lokal
Git	-	Mengelola dan mengunduh repository

Tabel 5.1 Prasyarat

5.2 Langkah Menjalankan Sistem

Tahapan deployment sistem dilakukan secara berurutan mulai dari pengambilan source code hingga verifikasi sistem berjalan dengan baik.

1. Clone Repository

Langkah pertama adalah mengunduh repository dari GitHub menggunakan perintah berikut:

```
git clone https://github.com/oliviadafina/distributed-sync-system.git  
cd distributed-sync-system
```

2. Build dan Menjalankan Container

Setelah repository berhasil diunduh, sistem dapat dijalankan menggunakan Docker Compose dengan perintah:

```
docker-compose up --build
```

Perintah tersebut akan melakukan beberapa proses secara otomatis, yaitu:

- Melakukan build Docker image dari file `Dockerfile.node`
- Mengunduh image Redis (`redis:7-alpine`)
- Menjalankan empat container utama, yaitu:
 - Node 1 pada port 8001
 - Node 2 pada port 8002
 - Node 3 pada port 8003
 - Redis pada port 6379
- Menjalankan mekanisme *health check* untuk memastikan Redis aktif sebelum node dijalankan

3. Verifikasi Sistem

Setelah seluruh container berjalan, langkah selanjutnya adalah melakukan verifikasi untuk memastikan sistem telah berjalan dengan benar. Pengguna dapat membuka browser dan mengakses beberapa endpoint berikut:

URL	Keterangan
http://localhost:8001/health	Status Node 1 dan informasi Raft
http://localhost:8002/health	Status Node 2 dan informasi Raft
http://localhost:8003/health	Status Node 3 dan informasi Raft
http://localhost:8001/docs	Dokumentasi API (Swagger UI)

Tabel 5.2 URL Endpoint

Node yang memiliki nilai `"raft_state": "leader"` merupakan node utama yang akan menangani permintaan terkait mekanisme lock.

4. Konfigurasi Autentikasi

Untuk mengakses endpoint yang memerlukan autentikasi, pengguna perlu memasukkan API Key melalui Swagger UI. Langkah-langkah:

- Buka halaman Swagger (`/docs`)
- Klik tombol Authorize
- Masukkan API Key `"admin-secret-key-123"`
- Klik Authorize, kemudian *Close*

Langkah ini perlu dilakukan pada setiap node yang digunakan.

5.3 Konfigurasi Environment

Konfigurasi sistem dilakukan melalui *environment variables* yang didefinisikan dalam file `docker-compose.yml`. Untuk melakukan penyesuaian konfigurasi, pengguna dapat menyalin file `.env.example` menjadi `.env` dengan perintah `"cp .env.example .env"`. Beberapa variabel penting yang dapat dikonfigurasi ditunjukkan pada Tabel berikut:

Variabel	Nilai Default	Keterangan
NODE_ID	node_1	Identitas unik setiap node
PEERS	otomatis	Daftar node lain dalam sistem
REDIS_HOST	redis	Host Redis dalam jaringan Docker
ELECTION_TIMEOUT_MIN	1500	Batas minimum election timeout (ms)
ELECTION_TIMEOUT_MAX	3000	Batas maksimum election timeout (ms)

HEARTBEAT_INTERVAL	500	Interval heartbeat dari Leader (ms)
--------------------	-----	-------------------------------------

Tabel 5.3 Variabel Konfigurasi

5.4 Pengujian Unit (Unit Testing)

Selain deployment menggunakan Docker, sistem juga menyediakan unit test yang dapat dijalankan secara lokal tanpa memerlukan container. Langkah menjalankan unit test:

```
pip install -r requirements.txt
python -m pytest tests/unit/ -v
```

Hasil yang diharapkan dari pengujian adalah seluruh test berhasil dijalankan tanpa error, seperti contoh berikut:

```
collected 62 items

test_cache_manager.py ..... PASSED
test_hash_ring.py ..... PASSED
test_lock_state.py ..... PASSED
test_raft_node.py ..... PASSED
test_security.py ..... PASSED

===== 62 passed =====
```

Sebanyak 62 unit test digunakan untuk menguji seluruh komponen utama sistem, termasuk cache, hashing, lock manager, Raft, dan keamanan.

5.5 Menghentikan Sistem

Untuk menghentikan sistem yang sedang berjalan, pengguna dapat menggunakan perintah “`docker-compose down`”. Perintah ini akan menghentikan seluruh container tanpa menghapus data. Jika ingin menghapus seluruh data dan melakukan reset sistem, gunakan perintah “`docker-compose down -v`”.

5.6 Referensi Port

Berikut adalah daftar port yang digunakan oleh masing-masing layanan dalam sistem:

Service	Port	URL
Node 1	8001	http://localhost:8001/docs
Node 2	8002	http://localhost:8002/docs
Node 3	8003	http://localhost:8003/docs
Redis	6379	redis-cli -p 6379
Locust	8089	http://localhost:8089

Tabel 5.4 Referensi Port

BAB 6

ANALISIS PERFORMA

Pengujian performa sistem dilakukan menggunakan *load testing tool* berbasis Python, yaitu Locust. Sistem dijalankan secara penuh menggunakan Docker Compose sebelum proses pengujian dimulai untuk memastikan kondisi lingkungan sesuai dengan skenario distribusi yang dirancang. Konfigurasi pengujian yang digunakan ditunjukkan pada Tabel dibawah ini:

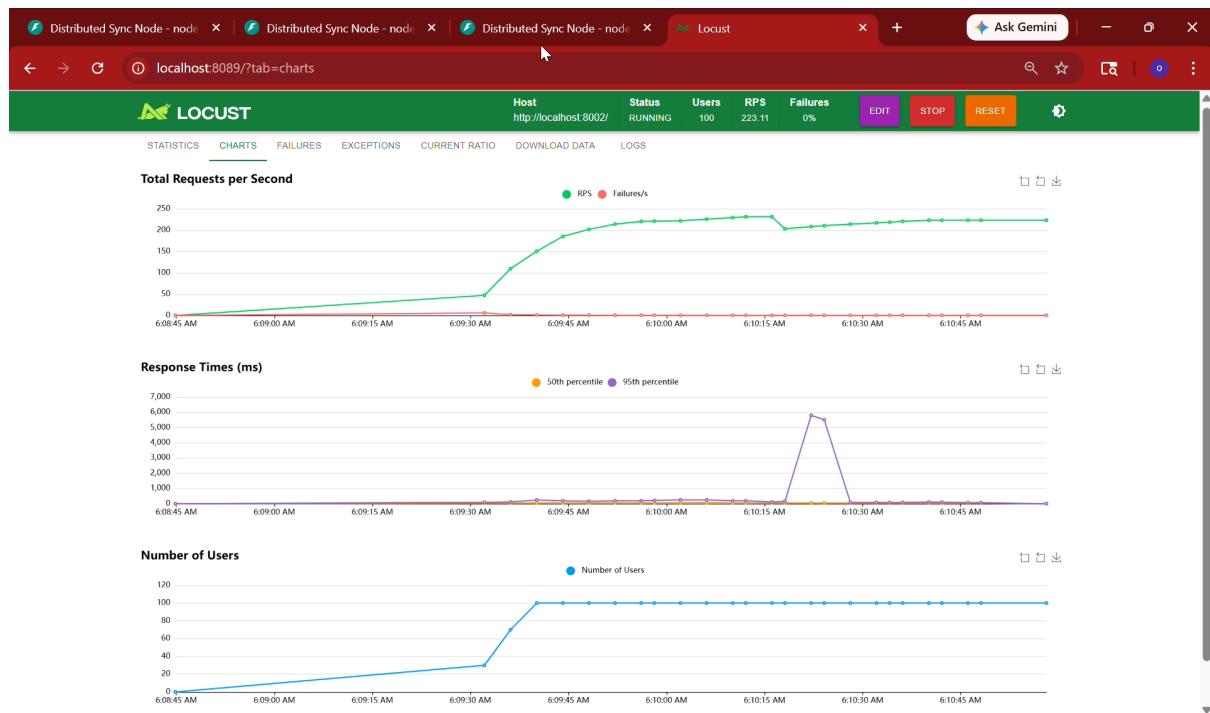
Parameter	Nilai
Tool	Locust v2.24
Jumlah Virtual Users	100
Ramp-up Rate	10 users/detik
Target Host	http://localhost:8002 (Node Leader)
Durasi Pengujian	±5 menit
Infrastruktur	3 Node Docker + 1 Redis
Skenario	Mixed workload (Lock, Queue, Cache)

Tabel 6.1 Konfigurasi Pengujian

Pengujian dilakukan dengan pendekatan *mixed workload* untuk mensimulasikan kondisi nyata, di mana berbagai jenis operasi dijalankan secara bersamaan.

6.1 Hasil Pengujian

Hasil pengujian divisualisasikan menggunakan dashboard Locust yang menampilkan metrik utama seperti throughput, response time, dan jumlah pengguna aktif.



Gambar 6.1 Hasil Pengujian dengan Locust

Throughput diukur menggunakan metrik Requests per Second (RPS) yang menunjukkan jumlah permintaan yang dapat ditangani sistem setiap detik.

Metrik	Nilai
RPS steady-state	~223 req/s
RPS maksimum	~250 req/s
Failure Rate	0%

Tabel 6.2 Hasil Throughput

Berdasarkan hasil pengujian, sistem menunjukkan peningkatan throughput yang stabil selama fase *ramp-up*, kemudian mencapai kondisi stabil pada sekitar 223 request per detik. Tidak ditemukan adanya kegagalan (*failure*) selama pengujian, yang menunjukkan bahwa sistem mampu menangani beban secara konsisten tanpa error.

Latency dianalisis menggunakan nilai persentil P50 (median) dan P95 untuk menggambarkan performa rata-rata dan kondisi terburuk.

Kondisi	P50	P95
Kondisi normal	< 50 ms	< 200 ms
Saat gangguan (election)	~500 ms	~5500 ms

Tabel 6.3 Analisis Latency

Terjadi lonjakan latency yang signifikan pada salah satu interval pengujian, di mana nilai P95 meningkat hingga sekitar 5500 ms. Kondisi ini bukan merupakan kesalahan sistem, melainkan akibat dari proses Raft Leader Election.

Proses tersebut terjadi ketika node tidak menerima *heartbeat*, sehingga memicu pemilihan leader baru. Selama proses ini berlangsung, permintaan sementara tertunda hingga leader baru terbentuk. Sistem mampu kembali stabil dalam waktu kurang dari 5 detik, yang menunjukkan kemampuan *self-healing* yang baik.

6.2 Analisis Per Komponen

1. Distributed Lock (Raft Consensus)

Metrik	Nilai
Latency (normal)	10 – 50 ms
Election overhead	1 – 5 detik
Konsistensi	Terjamin

Tabel 6.4 Analisis Distributed Lock (Raft Consensus)

Penggunaan Raft menyebabkan adanya tambahan latency karena proses replikasi log ke mayoritas node. Namun, hal ini memberikan jaminan konsistensi yang tinggi dalam pengelolaan lock.

2. Distributed Queue (Consistent Hashing)

Metrik	Nilai
Publish latency	< 20 ms
Forward latency	< 35 ms
Poll latency	< 10 ms

Tabel 6.5 Analisis Distributed Queue (Consistent Hashing)

Latensi tambahan akibat proses *forwarding* antar node relatif kecil, karena komunikasi terjadi dalam jaringan internal Docker dengan latensi rendah.

3. Cache Coherence (MESI Protocol)

Metrik	Nilai
Cache hit	< 5 ms
Cache miss	20 – 50 ms
Eviction	< 1 ms

Tabel 6.6 Analisis Cache Coherence (MESI Protocol)

Cache memberikan peningkatan performa signifikan, terutama pada kondisi *cache hit* yang dapat diakses langsung dari memori lokal tanpa melibatkan jaringan.

6.3 Perbandingan Single Node dan Distributed System

Aspek	Single Node	Distributed System
Fault Tolerance	Tidak tersedia	Toleran terhadap kegagalan node
Konsistensi Lock	Lokal	Konsisten global (Raft)
Skalabilitas Queue	Terbatas	Terdistribusi
Cache Coherence	Tidak ada	Sinkron antar node
Throughput	~200 RPS	~223 RPS
Recovery	Tidak ada	Otomatis (< 5 detik)
Kompleksitas	Rendah	Tinggi

Tabel 6.7 Perbandingan Sistem

Hasil menunjukkan bahwa sistem terdistribusi memiliki keunggulan signifikan dalam hal keandalan dan skalabilitas, meskipun dengan kompleksitas yang lebih tinggi.

6.4 Identifikasi Bottleneck dan Mitigasi

Bottleneck	Dampak	Mitigasi
Leader election	Lonjakan latency	Randomized timeout
Lock harus ke leader	Extra hop	Redirect ke leader
Broadcast cache	Overhead komunikasi	Masih optimal untuk 3 node

Redis sebagai pusat data	Risiko single point	Persistence & backup
--------------------------	---------------------	----------------------

Tabel 6.8 Bottleneck Sistem

Beberapa bottleneck teridentifikasi selama pengujian, namun telah diantisipasi melalui mekanisme mitigasi yang sesuai dengan prinsip desain sistem terdistribusi.

6.5 Kesimpulan Performa

Berdasarkan hasil pengujian yang telah dilakukan, sistem menunjukkan performa yang baik dengan karakteristik sebagai berikut:

1. Sistem mampu menangani hingga ± 223 request per detik tanpa error.
2. Sistem memiliki kemampuan *self-healing* dengan waktu pemulihan kurang dari 5 detik.
3. Latensi tetap rendah pada kondisi normal, terutama pada operasi cache dan queue.
4. Konsistensi data tetap terjaga selama pengujian tanpa ditemukan konflik atau inkonsistensi.

Secara keseluruhan, sistem berhasil memenuhi tujuan utama sebagai *distributed synchronization system* yang stabil, konsisten, dan scalable.